Assignment 5 – CO5

Reliable Software with Runtime Polymorphism and Exceptions

Field	Details
Name / Register No.	Kaviya V/192421022
GitHub repository	https://github.com/kaviyavellaisamy/DSA0107---C-/tree/main

Case study: hospital billing and insurance claims. All code compiles with g++ -std=c++17 -Wall -Wextra; sanitizer runs add -fsanitize=address,undefined. Pseudocode uses the layout ALGORITHM / INPUT / OUTPUT / BEGIN … END with ← for assignment; adjust headings to match your copy of the SIMATS pseudocode standard. Workflow: Understand → Abstract → Design → Implement → Break → Measure → Interpret. Money is kept in rupees as double only for the small demo figures; a production system should use integer paise.

PART A – DESIGN (25 marks)

Task A1 – Design the Polymorphic Bill Hierarchy (7 marks)

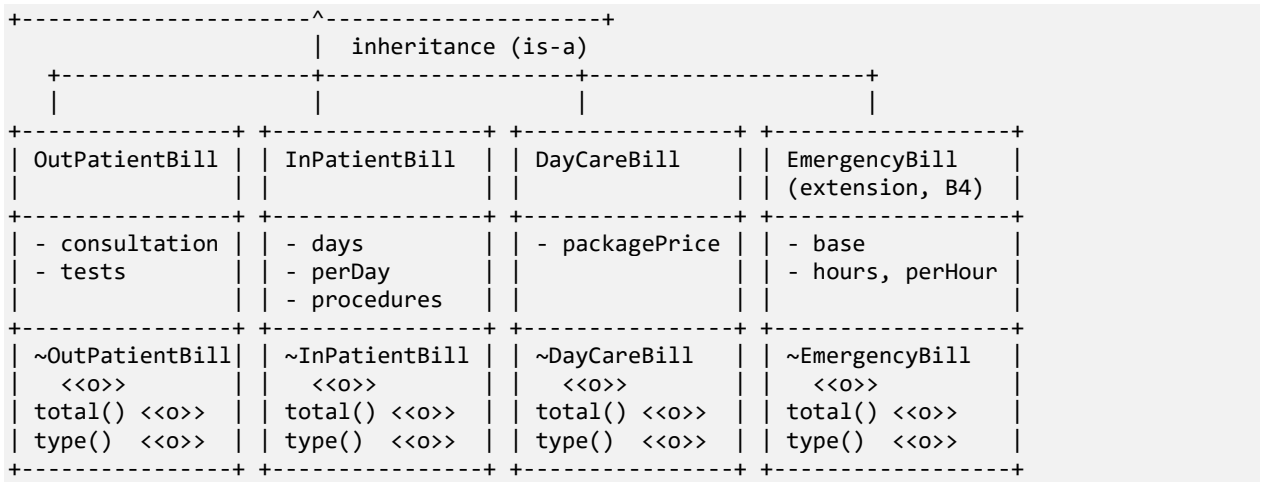
A1.1 – Common interface and common data (2 marks)

Item	Why every bill needs it
total() const – returns the amount payable	The one operation the day's processing needs; each bill kind computes it differently, so it is pure virtual and noexcept (it only does arithmetic).
type() const – returns "OP", "IP", "DC", "ER"	Used by printing and by payers that price by bill type (government scheme).
print() const – shows type, patient and total	Identical for all bills, so it is a normal non-virtual function in Bill that calls the two virtual functions (template-method pattern).
virtual destructor	Bills are deleted through Bill*; without it the derived part is never destroyed (A2.3, B1.4).
Common data: patient name	Stored once in Bill (protected). Kind-specific data (days, rate, package) lives in the derived classes.

A1.2 – The hierarchy (3 marks)

UML class diagram ({abstract} = has a pure virtual function; <<pv>> pure virtual, <<v>> virtual, <<o>> overrides)

```
+-----------------------------------------+
|           Bill   {abstract}             |
+-----------------------------------------+
| # patient : string                      |
+-----------------------------------------+
| + ~Bill()                      <<v>>    |
| + total() : double             <<pv>>   |
| + type()  : string             <<pv>>   |
| + print()          non-virtual (calls total, type)
```



Function	Kind in Bill	In derived classes
total()	pure virtual (= 0)	overridden – each kind has its own formula
type()	pure virtual (= 0)	overridden
~Bill()	virtual	each derived destructor overrides it
print()	ordinary (non-virtual)	inherited unchanged; calls total() and type() through the vtable

A1.3 – Process all bills together (2 marks)

```
ALGORITHM DayTotal
INPUT  bills : array of n pointers to Bill   (any mix of bill kinds)
OUTPUT sum   : the total of every bill for the day
BEGIN
    sum ← 0
    FOR i ← 0 TO n - 1 DO
        sum ← sum + bills[i].total()      // run-time binding picks the right total()
    END FOR
    RETURN sum
END
```

The loop never mentions a kind of bill, so a new kind is processed with no change to this algorithm.

Alternative Concept and Complexity Analysis – one Bill struct with a type field and a switch

```
ALGORITHM Total(b)
BEGIN
  SWITCH b.type
    CASE OP: RETURN b.a + b.b
    CASE IP: RETURN b.days * b.a + b.c
    CASE DC: RETURN b.a
    CASE ER: RETURN b.a + b.hours * b.b
  END SWITCH
END
```

```
ALGORITHM DayTotal
BEGIN
  sum ← 0
  FOR each bill b DO sum ← sum + Total(b)
  RETURN sum
END
```

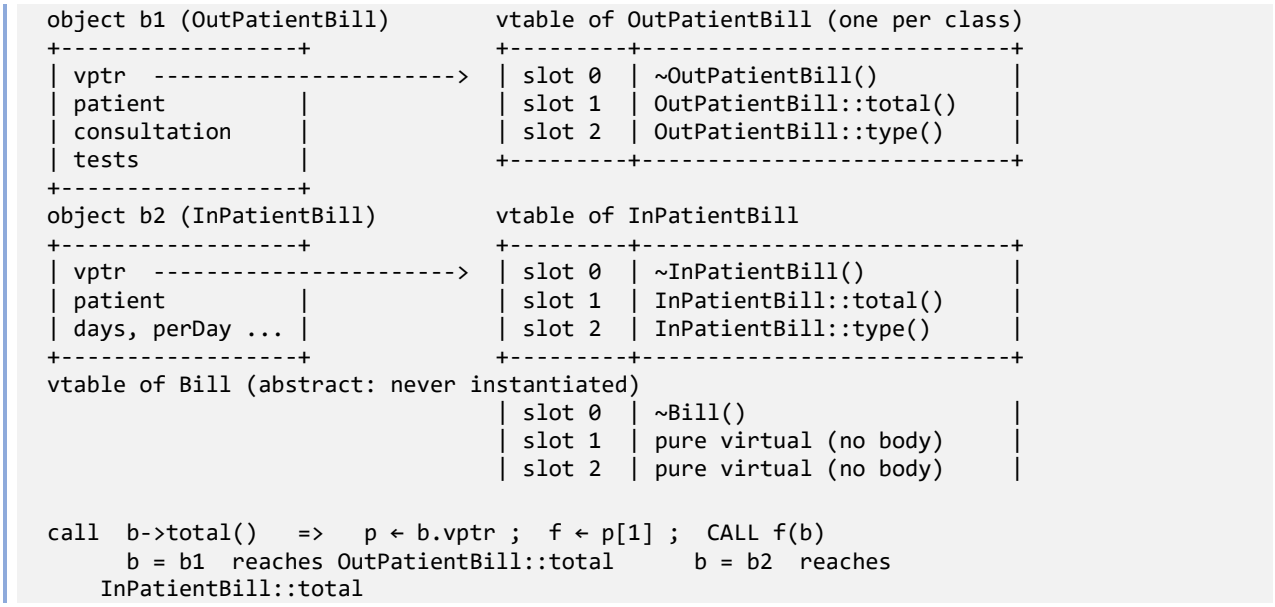
Aspect	Abstract class + virtual	Struct + type field + switch
Time to total n bills	O(n): one indirect call per bill	O(n): one jump-table branch per bill (measured in B3: switch ≈ 20–25% faster)
Space per bill	vptr (8 B) + only the fields of that kind: OP 24 B, IP 32 B, DC 16 B, ER 24 B (measured sizeof)	every bill carries the union of all fields + tag: 32 B for every kind; wasted for small kinds
Code changed to add a new bill type	one new class; no existing class or loop edited	new enum value + a new case in EVERY switch (total, type, print …) + new fields in the struct; all existing code re-tested
Forgotten case	compile error (pure virtual not overridden)	silently wrong unless -Wswitch is noticed

Task A2 – Understand Runtime Binding (5 marks)

A2.1 – Early and late binding (2 marks)

Early (compile-time) binding: the compiler decides which function a call reaches from the static type of the pointer. For a non-virtual function, b->print() with Bill* b is always Bill::print() – the address is fixed in the machine code. **Late (run-time) binding:** for a virtual function, b->total() is compiled to "load b's vptr, load slot k of that vtable, call it". The static type Bill* only fixes the slot number; the dynamic type of the object (OutPatientBill, InPatientBill …) decides which function sits in that slot, so the call is resolved when the program runs.

A2.2 – The virtual tables (2 marks)



A2.3 – Why Bill needs a virtual destructor (1 mark)

delete b with a Bill* is a static decision when ~Bill is not virtual: only ~Bill runs, and the derived destructor and its members (the vector of procedure records in InPatientBill) are never destroyed. The

C++ standard makes this undefined behaviour. With a virtual ~Bill the delete goes through the vtable (slot 0), runs ~OutPatientBill first, then ~Bill, and the correct amount of memory is freed.

Predict the Output – given program

```
#include <iostream>
struct Bill { virtual ~Bill() { std::cout<< "~Bill "; } virtual double total() const =
      0; void show() const{ std::cout<<total() << ' '; } };
struct OPBill : Bill { ~OPBill() override { std::cout<< "~OPBill "; } double total()
      const override { return 500; } };
int main(){ Bill* b = new OPBill; b->show(); delete b; std::cout<< '\n'; }
```

My predicted output (hand-traced; write your own before running): 500 ~OPBill ~Bill

Actual output: 500 ~OPBill ~Bill – prediction and result agree.

How show() reached OPBill::total(): show() is not virtual, so b->show() calls Bill::show() directly. Inside it, total() really means this->total() where this is a Bill* that points to an OPBill object. total() is virtual, so the call goes through the object's vptr to OPBill::total() and prints 500. The destructor chain runs derived first (~OPBill) then base (~Bill).

Experiment – remove virtual from ~Bill

Predicted: delete b calls only ~Bill (static type), so the output becomes 500 ~Bill. **Actual:** 500 ~Bill – ~OPBill never ran. g++ warns: "deleting object of abstract class type 'Bill' which has non-virtual destructor will cause undefined behavior [-Wdelete-non-virtual-dtor]". Under -fsanitize=address the program ran to completion with exit code 0 and no report, because OPBill has no data members: the object has the same size as Bill, so the wrong-size delete is invisible to the sanitizer. This shows that "no sanitizer message" is not proof of correctness – the behaviour is still undefined.

To make the damage visible, give the derived class an owned resource (a std::vector of 100 strings) and repeat: AddressSanitizer reports **new-delete-type-mismatch** ("size of the allocated type: 32 bytes; size of the deallocated type: 8 bytes"). With that check switched off, LeakSanitizer reports the vector's heap block as leaked because ~IPBill never ran. With virtual ~Bill the run is clean.

Alternative Concept and Complexity Analysis – non-virtual total() and a dynamic_cast chain

```
ALGORITHM TotalOf(b)                        // b : pointer to Bill
BEGIN
    IF b IS-A OutPatientBill  THEN RETURN cast(b).consultation + cast(b).tests
    ELSE IF b IS-A InPatientBill THEN RETURN cast(b).days * cast(b).perDay +
      cast(b).procedures
    ELSE IF b IS-A DayCareBill   THEN RETURN cast(b).price
    ...                                       // one test per bill kind, k kinds
END
```

k bill kinds (2,000,000 bills, -O2, measured)	virtual call	dynamic_cast chain	ratio
k = 2	17.4 ms (8.7 ns/bill)	43.8 ms (21.9 ns/bill)	2.5×
k = 4	22.1 ms (11.0 ns/bill)	84.1 ms (42.0 ns/bill)	3.8×
k = 8	25.0 ms (12.5 ns/bill)	163.9 ms (81.9 ns/bill)	6.5×

Time per bill: virtual call = O(1) (one vtable load + indirect call); cast chain = O(k) in the worst case (up to k type checks, each walking RTTI), growing with every bill type the hospital adds. Space: the virtual design stores one 8-byte vptr in each object (measured: a class with one double is 8 bytes without

virtual functions and 16 bytes with them) – the chain needs the vptr as well, because dynamic_cast only works on polymorphic classes, so it saves no space. Every new bill kind also forces an edit of the chain.

Task A3 – Design with Abstract Classes (5 marks)

A3.1 – Payer interface (3 marks)

```
+-------------------------------------------+
|  Payer {abstract}                         |
|  + ~Payer()                      <<v>>    |
|  + amountCovered(b : Bill&) : double <<pv>>  |
|  + name() : string               <<pv>>   |
+----------------------^--------------------+
                       |
   +-----------+-------+------+-----------------+
   |           |              |                 |
+----------+ +--------------+ +----------------+ +----------------+
| SelfPay  | | Insurance    | | Government-    | | CorporateTieUp |
|          | | - coverLimit | | Scheme         | | (extension)    |
| covers 0 | | - copay %    | | - rate per     | | - percent      |
|          | | covers       | |   bill type    | | covers         |
|          | | min(limit,   | | covers min(    | | total*%/100    |
|          | | total*(100-  | | total, package | |                |
|          | |  copay)/100) | | rate)          | |                |
+----------+ +--------------+ +----------------+ +----------------+
```

```
ALGORITHM AmountCovered for Insurance(bill)
INPUT  bill with total T; coverLimit; copay (percent)
OUTPUT amount the insurer pays
BEGIN
    wanted ← T * (100 - copay) / 100
    RETURN MIN(coverLimit, wanted)
END

ALGORITHM AmountCovered for GovernmentScheme(bill)
BEGIN
    IF bill.type has no fixed rate THEN RETURN 0
    RETURN MIN(bill.total(), rate[bill.type()])
END
```

A3.2 – Extension without changing any class (2 marks)

A corporate tie-up is one new class, CorporateTieUp, derived from Payer, that overrides amountCovered as total × percent / 100. The settlement algorithm only calls p.amountCovered(bill) through a Payer reference, so Bill, SelfPay, Insurance, GovernmentScheme and settle() are untouched (open–closed principle). B4 implements this and re-runs every earlier test unchanged: 27/27 pass.

Alternative Concept and Complexity Analysis – function pointers / std::function

```
TYPE PayerPolicy ← FUNCTION(total) RETURNS amount
policy ← LAMBDA(total) → MIN(limit, total * (100 - copay) / 100)
        // limit, copay CAPTURED
covered ← policy(bill.total())
```

Aspect (measured, 20,000,000 calls, -O2)	abstract Payer	std::function
Time per call	O(1): 59.5 ms (2.98 ns)	O(1): 59.8 ms (2.99 ns) – same

Aspect (measured, 20,000,000 calls, -O2)	abstract Payer	std::function
Space per payer	object 24 B (vptr + limit + copay) + 1 shared vtable	std::function 32 B; the captured limit and copay (16 B) live inside it
Where the limit is stored	named data members, checked in the constructor, can be read with a getter (limit())	hidden inside the closure; cannot be inspected, printed or validated afterwards
Several operations (name(), limit())	extra virtual functions	would need one std::function per operation, or a struct of them
Best for	payers with several related operations and validated data	one small, throw-away policy chosen at run time

Task A4 – Design the Exception Strategy (4 marks)

A4.1 – The errors and the exception hierarchy (2 marks)

Error	Detected in	Exception thrown
Negative charge / days / limit, invalid co-payment	bill and payer constructors (nonNegative)	InvalidAmountError
Claim above the policy cover	approveClaim()	ClaimError
Unknown package code	DayCareBill::lookup()	UnknownPackageError
Missing patient record	PatientRegistry::idOf()	RecordNotFoundError
Memory allocation failure	operator new / std::vector	std::bad_alloc (from the library – not wrapped)
Text where a number is expected	std::stod	std::invalid_argument (library)

```
std::exception
 ├── std::bad_alloc                      (library)
 ├── std::logic_error ── std::invalid_argument, std::out_of_range   (library)
 └── std::runtime_error
       └── BillingError                  (application base: catch this for “any
     billing problem”)
               ├── InvalidAmountError
               ├── ClaimError
               ├── UnknownPackageError
               └── RecordNotFoundError
```

A4.2 – Exception specifications (2 marks)

Function	May throw?	Reason
Bill::total(), derived total()	noexcept	pure arithmetic on already-validated members; lets callers and containers rely on it
patient() accessor, destructors	noexcept	cannot fail; destructors are noexcept by default
constructors, Insurance(…), DayCareBill(…)	may throw	they validate input; a throwing constructor leaves no half-built object

Function	May throw?	Reason
approveClaim(), settle(), idOf()	may throw	report ClaimError, BillingError, RecordNotFoundError to the caller who can decide

Why destructors must not throw: during stack unwinding the runtime is already handling one exception and destroys local objects; if a destructor then throws a second exception that escapes, std::terminate is called and the program ends with every other resource unreleased. Destructors are therefore noexcept (the default since C++11) and must catch and log their own errors.

Alternative Concept and Complexity Analysis – return codes

```
ENUM Status ← { OK, NEGATIVE, ABOVE_COVER, UNKNOWN_PACKAGE, NO_RECORD }
ALGORITHM Approve(billed, cover, OUT paid) RETURNS Status
BEGIN
    IF billed < 0 OR cover < 0 THEN RETURN NEGATIVE
    IF billed > cover          THEN RETURN ABOVE_COVER
    paid ← billed ; RETURN OK
END
caller:  s ← Approve(b, c, paid) ; IF s ≠ OK THEN (handle OR return s upward) ; …
        // after EVERY call
```

Aspect	Exceptions	Return codes
Time, normal path	O(1) per call, no cost when nothing is thrown (measured B2/B3: same speed at 0% invalid)	O(1): one compare-and-branch after every call
Time, error path	O(depth) unwinding, ≈ 1.4 µs per throw (measured)	O(1) per level to pass the code up
Extra space / code	none in the normal path; a hierarchy of classes; handler tables in the binary	an out-parameter or result struct for every function, and a check (and a forward) after every call; one forgotten check silently loses the error
Constructors	can signal failure	cannot return a code

Task A5 – Design Recovery and Rethrowing (4 marks)

A5.1 – Where to catch (2 marks)

Exception	Caught / logged where	Action
InvalidAmountError (from a constructor)	the data-entry / import function that built the bill	reject that one bill with a message, continue with the next
ClaimError	settle(): catch, log “audit: …”, rethrow with throw;	audit trail written where the facts are known; the decision stays with the caller
UnknownPackageError, RecordNotFoundError	the request handler (one patient request)	report the reason to the clerk; no state to undo because of RAII

Exception	Caught / logged where	Action
std::bad_alloc and anything unexpected	top level of main(): catch (const std::exception&) then catch (...)	report to the user, keep the service running or exit cleanly; all objects are already destroyed

```
ALGORITHM Settle(bill, insurer)
BEGIN
    TRY    paid ← Approve(bill, insurer)
    CATCH ClaimError AS e
        WRITE "audit: ", e.message
        RETHROW                        // same exception object, same dynamic type
    END TRY
    RETURN paid
END
```

A5.2 – No leaks on errors: stack unwinding and RAII (2 marks)

When throw executes, the runtime searches outward through the call chain for a matching handler. Before control reaches it, every function frame between the throw and the handler is left, and the destructor of every fully constructed local object in it runs, in reverse order of construction. This is stack unwinding. An object that owns a resource (std::vector, std::unique_ptr, a file stream, a Guard class) releases it in its destructor, so memory and files are freed on every path – normal return or exception – with no explicit cleanup code. In the B2 demonstration, an InPatientBill constructed between two Noisy objects was destroyed in the order ~B, ~IP, ~Bill(Meena), ~A while a ClaimError passed through.

Predict the Output – given program

```
#include <iostream>
#include <string>
struct Guard { std::string n; explicit Guard(const std::string& s) : n(s) {} ~Guard()
      { std::cout<< '~' << n << ' '; } };
void f(){ Guard g("f"); throw 42; }
int main(){ try { Guard m("main"); f(); std::cout<< "not printed "; } catch (int x) {
      std::cout<< "caught " << x; } catch (...) { std::cout<< "other"; } std::cout<<
      '\n'; }
```

My predicted output (hand-traced; write your own before running): ~f ~main caught 42

Actual output: ~f ~main caught 42 – agrees. **Order:** throw 42 leaves f(), so its local Guard g("f") is destroyed first (~f); the exception then leaves the try block, destroying Guard m("main") (~main) – the reverse of construction order and innermost first; only then does the catch (int) handler run and print “caught 42”. “not printed” is skipped because a throw abandons the rest of the try block immediately; control never returns to the statement after f(). The int handler is chosen before catch (...) because handlers are tried in order and the type matches exactly.

Alternative Concept and Complexity Analysis – delete in every catch block

```
ALGORITHM Process()                    // manual cleanup
BEGIN
    a ← NEW Bill ; b ← NEW Record
    TRY    Step1(a) ; Step2(b)
    CATCH ErrorType1  DELETE a ; DELETE b ; RETHROW
    CATCH ErrorType2  DELETE a ; DELETE b ; RETHROW      // copy of the cleanup again
    CATCH ANY         DELETE a ; DELETE b ; RETHROW
    END TRY
    DELETE a ; DELETE b                                   // normal path
```

```
END
        // RAII version: a and b are owners → no CATCH needed
```

Aspect	delete in every catch	RAII (unique_ptr / vector)
Time	same O(1) per resource	same
Code size	(number of catch blocks + 1) copies of the cleanup for each resource	zero cleanup lines; one destructor written once
Leak places when a new error path is added	every new catch (or early return / new throw site) is a new place where a delete can be forgotten: with 2 resources and 3 handlers, 8 delete statements must all be right	none – the destructor runs on the new path automatically
Exception thrown inside a catch	skips the remaining deletes → leak	still safe

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION (25 marks)

The complete billing system is in two header files (core.h: exceptions, bills, payers, registry; ext.h: the B4 extension) with a test file that prints expected and actual values. core.h was not modified when the extension was added (its checksum is the same before and after).

Task B1 – Implement and Validate the Polymorphic Billing (7 marks)

B1.1 – Validation: every bill kind with every payer (2 marks)

Test data: OP Ravi (consultation 500 + tests 1200), IP Meena (3 days × 2500 + procedures 40000), DC Kiran (CATARACT package 18000). Payers: SelfPay; Insurance (limit 40000, co-payment 10%); Government (OP 800, IP 30000, DC 18000). Every expected value was computed by hand first.

Bill	Total (hand)	SelfPay covers / patient pays	Insurance covers / patient pays	Government covers / patient pays
OP 500+1200	1700	0 / 1700	1530 (90% of 1700) / 170	800 / 900
IP 3×2500+40000	47500	0 / 47500	min(40000, 42750) = 40000 / 7500	30000 / 17500
DC CATARACT	18000	0 / 18000	16200 (90% of 18000) / 1800	18000 / 0
Day total (all three, via Bill*)	81200			

Result: all 20 core checks PASS (see screenshot and the 27/27 summary after the extension).

B1.2 – Comparison with the Part A design (1 mark)

- Added a protected-copy rule: Bill's copy constructor and assignment are deleted, because copying a Bill through a base reference would slice it (A1 did not mention this).
- InPatientBill owns a vector of procedure records so that the virtual destructor has something real to destroy and B1.4 can be demonstrated.
- Validation moved into the constructors (helper nonNegative) so no invalid bill can exist; in the design this was only "checked somewhere".
- Insurance gained wanted() and limit() because approveClaim() needs the amount before capping it; amountCovered() still caps at the cover limit.
- A g_alive counter was added to prove that no Bill is leaked.

B1.3 – Prove runtime binding (2 marks)

Experiment 1 – remove only the word virtual from the pure declaration (virtual double total() const = 0 → double total() const = 0): the compiler refuses. **Errors received:** "initializer specified for non-virtual method 'double Bill::total() const'" (a "= 0" is only allowed on a virtual function) and, for each derived class, "'double OutPatient::total() const' marked 'override', but does not override".

Experiment 2 – make the base function an ordinary function with a body (double total() const { return 0; }) and remove override from the derived ones so that it compiles. **Output changes** from OP Ravi: 1700 / IP Meena: 47500 / Total 49200 to **OP Ravi: 0 / IP Meena: 0 / Total 0**: print() and the loop call Bill::total() because the call is bound at compile time to the static type Bill*. The derived formulas are never reached – exactly the failure that virtual prevents.

B1.4 – Prove the virtual destructor (2 marks)

Run (-fsanitize=address -g)	Result
Derived class owns a vector of 100 strings, ~Bill NON-virtual	ERROR: AddressSanitizer: new-delete-type-mismatch – "size of the allocated type: 64 bytes; size of the deallocated type: 40 bytes"; the run aborts.
Same, with new_delete_type_mismatch=0 to see the consequence	LeakSanitizer: 4900 bytes leaked in 101 allocations (3200 bytes direct + 1700 bytes in 100 indirect string objects): the vector inside the derived object was never destroyed.
~Bill virtual	prints 47500, exit code 0, no sanitizer report.

Complete the Code – TODO 1 to TODO 3 (completed)

```
// TODO 1
virtual double total() const = 0;
virtual std::string type() const { return "Bill"; }

// TODO 2 (in OutPatient)
double total() const override { return consultation + tests; }
std::string type() const override { return "OP"; }

// TODO 3 (in InPatient)
double total() const override { return days * perDay + surgery; }
std::string type() const override { return "IP"; }
```

Executed output (matches the expected output): OP Ravi: 1700 | IP Meena: 47500 | Total 49200.

Alternative Concept and Complexity Analysis – templates, one container per bill type

```
ALGORITHM DayTotal(opList, ipList, dcList, erList)
        // one typed container per kind
BEGIN
    sum ← 0
    FOR each b IN opList DO sum ← sum + b.consultation + b.tests
        // inlined, no indirect call
    FOR each b IN ipList DO sum ← sum + b.days * b.perDay + b.procedures
    ...
        // one loop per kind
END
```

Aspect (10,000,000 bills, -O2, measured)	virtual through Bill*	typed containers (no virtual call)
Time	O(n): 118.0 ms (pointer chase + indirect call)	O(n): 42.1 ms (about 2.8× faster: contiguous memory, inlined arithmetic)
Space	n × 8-byte pointer + n separately heap-allocated objects	objects stored directly in the vectors, no pointers; one container per type (4 here)
What is lost	nothing – one list holds every kind	no single list of all bills: order of entry is lost, every algorithm must be written once per type (or as a template), the set of types is fixed at compile time and a new type needs a new container and edits to every loop

Both give the same total (checked). Templates win on speed when the kinds are separate; virtual functions win when bills arrive mixed and new kinds appear after compilation of the calling code.

Task B2 – Engineer Error Conditions (5 marks)

Deliberately constructed error conditions (observed)

Test input	Exception observed (actual output)	What it reveals
OutPatientBill with consultation −500	InvalidAmountError: negative consultation	validation in the constructor means no invalid bill exists
Claim: bill 1700, insurer share 1530 > cover 1000	ClaimError: claim exceeds policy cover of 1000	a business-rule error has its own type, so it can be audited separately
Package code "VALVE"	UnknownPackageError: unknown package VALVE	the message carries the offending value
std::stod("abc")	std::invalid_argument: stod	library exception caught by the std::exception handler – the hierarchy sits under std::exception
Patient “Meena” not in registry	RecordNotFoundError: no record for patient Meena	lookup failure is reported, not hidden as a default value
std::vector<double>(2^60 elements)	std::bad_alloc: std::bad_alloc	allocation failure is catchable; nothing was constructed
new double[2^60] (size in a variable)	std::bad_alloc: std::bad_array_new_length	a more specific subclass of bad_alloc; the handler for the base catches it

Note on the sanitizer: AddressSanitizer itself aborts a program that asks for an absurdly large block ("requested allocation size … exceeds maximum supported size") instead of letting new throw. Therefore the bad_alloc tests are run in the normal build, and the same program is run with the two allocation tests compiled out (-DNO_BIG) under -fsanitize=address,undefined: identical output, exit code 0, no sanitizer message.

catch(...) at the top level keeps the service running

```
request 1:
  +A
  +B
  ~B
~IP ~Bill(Meena)   ~A
  caught std::exception: claim rejected midway
  bills alive after request 1: 0
request 2:
  caught unknown exception (catch-all)
        <- throw 7 (an int, not a std::exception)
  bills alive after request 2: 0
program still running; bills alive at end: 0
```

The ClaimError thrown in request 1 passed through a frame holding two Noisy objects and one InPatientBill; every destructor ran in reverse order (B, then the InPatientBill, derived part before base part, then A). A thrown int was caught only by catch (...), the loop went on to the end, and the counter of live bills stayed at 0. **Interpretation:** the exception hierarchy lets specific errors be handled specifically while catch (...) is only the last safety net; RAII made the cleanup automatic.

Predict the Output – given program

```
#include <iostream>
#include <stdexcept>
void validate(double amount){ if (amount < 0) throw std::invalid_argument("negative");
      if (amount > 1e6) throw std::out_of_range("too large"); std::cout<< "ok "; }
void process(double amount){ try { validate(amount); } catch (const std::out_of_range&
      e) { std::cout<< "[log " <<e.what() << "] "; throw; } }
int main(){ for (double a : {100.0, -5.0, 2e6}) { try { process(a); std::cout<< "done
      "; } catch (const std::exception& e) { std::cout<< "caught " <<e.what() << ' ';
      } } std::cout<< '\n'; }
```

My predicted output (hand-traced; write your own before running): ok done caught negative [log too large] caught too large

Actual output: ok done caught negative [log too large] caught too large – agrees. Which handler caught what: 100.0 passes both checks, prints "ok", and process returns normally so "done" follows. −5.0 throws invalid_argument; process catches only out_of_range, so the exception passes straight through to main, where catch (const std::exception&) catches it (invalid_argument derives from logic_error derives from exception) → "caught negative", and "done" is skipped. 2e6 throws out_of_range; process's handler matches, logs "[log too large]", and throw; rethrows the same object, which main catches → "caught too large". Only one exception is logged because only out_of_range has a handler in process.

Complete the Code – TODO 1 and TODO 2 (completed)

```
    // TODO 1  (inside approve)
    if (billed < 0 || cover < 0) throw std::invalid_argument("negative amount");
    if (billed > cover) throw ClaimError("exceeds cover");
    return billed;
```

```
    // TODO 2  (inside the catch of settle)
    std::cout << "audit: " << e.what() << '\n';
    throw;
```

Executed output (matches the expected output): approved 40000 | audit: exceeds cover | claim rejected: exceeds cover | error: negative amount.

Replace throw; with throw e;

Experiment: a more specific exception LimitExceeded (derived from ClaimError, extra member overBy = 7000) is thrown and passed through an audit handler that catches const ClaimError&.

Version	Observed	Explanation
throw;	main caught LimitExceeded (dynamic type kept), overBy=7000; the address of the exception object is the same	rethrows the original exception object with its full dynamic type
throw e;	main caught only ClaimError (the LimitExceeded part was sliced off); the object address differs	throw e; throws a NEW object copy-constructed from the static type of e (ClaimError): the derived part is sliced off and a copy is made

In the assignment's program, with throw e; the output is the same text, because the handler in main catches ClaimError either way; the difference appears as soon as a derived type or extra data is thrown.

Alternative Concept and Complexity Analysis – std::optional<double> instead of exceptions

```
ALGORITHM Approve(billed, cover) RETURNS OPTIONAL amount
BEGIN
    IF billed < 0 OR cover < 0 OR billed > cover THEN RETURN NOTHING
        // no reason given
    RETURN billed
END
caller:  r ← Approve(b, c) ; IF r HAS VALUE THEN use(r) ELSE reject
```

1,000,000 claims, -O2 (measured)	return code / optional	exceptions
0% invalid	return code 2.66 ms, optional 2.61 ms	2.13 ms (no penalty on the normal path)
1% invalid	return code 2.02 ms, optional 2.98 ms	24.56 ms (≈ 12× slower)
50% invalid	return code 7.42 ms, optional 7.15 ms	879.0 ms (≈ 118× slower)
Space per result	optional<double> = 16 bytes instead of 8 (value + flag)	none on the normal path; the exception object (≈ 1.4 µs per throw) only exists while thrown
Information	only "rejected", no reason; every caller must check	a type and message carried to the right handler automatically

Time complexity: both O(n) for n claims, but the cost per rejected claim is a constant of a few ns for optional versus about 1.4 µs for a throw; the optional answer is cheaper when rejections are common, the exception answer is cleaner when they are rare.

Task B3 – Virtual Functions vs switch vs Exceptions: The Cost (8 marks)

B3.1 – Correctness on 10 bills (2 marks)

Ten bills (4 OP, 3 IP, 3 DC) were totalled with virtual functions and with a single struct + type field + switch. Hand check: 1500 + 3550 + 18000 + 1950 + 11200 + 18000 + 2400 + 18850 + 18000 + 2850 = 96300. **Output: virtual total = 96300, switch total = 96300, agree: yes.** For the large runs the variant version also agreed with both (the program prints the agreement check for each size).

B3.2 – Increasing data size (2 marks) and the std::variant alternative

Bills (4 kinds, random mix, -O2)	virtual (Bill*)	struct + switch	std::variant + std::visit
1,000	0.0039 ms	0.0029 ms	0.0029 ms
1,000,000	14.30 ms	10.72 ms	10.44 ms
10,000,000	145.9 ms	112.4 ms	99.9 ms

Space per element (measured sizeof): virtual: 8-byte pointer + heap object of 24 / 32 / 16 / 24 bytes (OP / IP / DC / ER) plus allocator overhead; switch struct: 32 bytes for every bill; variant: 32 bytes for every bill = the largest alternative (24 B) + the tag, with the object stored in place (no pointer, no heap allocation). Timing note: single runs on a shared cloud machine; use your own repeated measurements for the report.

B3.3 – Cost of exceptions (2 marks)

1,000,000 bills validated	return codes	exceptions	ratio
0% invalid	2.66 ms	2.13 ms	≈ 1× (equal within noise)
1% invalid (10,000 throws)	2.02 ms	24.56 ms	≈ 12×
50% invalid (500,000 throws)	7.42 ms	879.0 ms	≈ 118×

Cost of one throw plus catch measured alone: about 1,400 ns.

B3.4 – Engineering trade-off (2 marks)

- **Virtual vs switch vs variant:** all are O(n). The switch and the variant were 20–30% faster than virtual calls on 1–10 million bills because their data is contiguous (the virtual version chases a pointer per bill). The difference is a few nanoseconds per bill; 200,000 bills a day cost under 3 ms in any of them. The deciding factor is therefore maintenance, not speed.
- **Use virtual functions** when new kinds are added often and by different people, kinds hold different data, or objects must be created from configuration; **use switch / variant** when the set of kinds is closed and small and the loop is performance-critical (variant also gives compile-time exhaustiveness through visit).
- **Exceptions** cost nothing on the normal path and about 1.4 µs per throw, so they are the right tool for rare, exceptional failures (1% or less: 12× on a 2 ms job is still only 22 ms per million bills). Use return codes or optional when failure is a common, expected outcome (50% rejects: 118× slower) or when the loop is a hot path.

Task B4 – Final Engineering Decision (5 marks)

B4.1 – Practical extension without changing existing classes (2 marks)

New bill type **EmergencyBill** (base charge + hours × hourly charge) and new payer **CorporateTieUp** (company pays a percentage of every bill) were added in a new header ext.h that includes core.h; no existing class was edited (checksum of core.h unchanged). The earlier test file is re-run unchanged on the same core, followed by the new tests.

```
// ext.h  -  EXTENSION: one new bill type + one new payer. core.h is NOT modified.
#pragma once
#include "core.h"
class EmergencyBill : public Bill {
        // base charge + time-based charge
    double base_, hours_, perHour_;
public:
    EmergencyBill(const std::string& p, double base, double hours, double perHour) :
      Bill(p), base_(nonNegative(base, "base charge")), hours_(nonNegative(hours,
      "hours")), perHour_(nonNegative(perHour, "hourly charge")) {}
    ~EmergencyBill() override { if (g_trace) std::cout << "~ER "; }
    double total() const noexcept override { return base_ + hours_ * perHour_; }
    std::string type() const override { return "ER"; }
};
class CorporateTieUp : public Payer {
        // company pays a fixed percentage of every bill
    double percent_;
public:
    explicit CorporateTieUp(double pct) : percent_(pct) { if (pct < 0 || pct > 100)
      throw InvalidAmountError("percentage must be 0..100"); }
    double amountCovered(const Bill& b) const override { return b.total() * percent_ /
      100; }
    std::string name() const override { return "Corporate"; }
};
```

New test (hand-verified)	Expected	Actual
ER total: 2000 + 5 × 800	6000	6000 PASS
ER / Corporate 70%: covered, patient pays	4200, 1800	4200, 1800 PASS
OP 1700 / Corporate 70%	1190	1190 PASS
ER / Insurance 90% of 6000	5400	5400 PASS
Day total with the unchanged loop over Bill*: 1700 + 6000	7700	7700 PASS
Corporate percentage 140 rejected	InvalidAmountError	InvalidAmountError PASS

All 20 earlier tests still pass: **27/27 checks passed, bills alive at end: 0.**

B4.2 – Resource safety (1 mark)

The 27 checks include thrown-and-caught exceptions (negative charge, unknown package, claim above cover, invalid percentage). Built with -fsanitize=address,undefined -g the run printed 27/27 passed, **exit code 0, no AddressSanitizer, LeakSanitizer or UBSan report**. The B2 error demonstration (7 error conditions, a top-level catch-all and 10 bills) is also clean under the same flags (0 bytes of sanitizer output, exit code 0).

B4.3 – Production decision (2 marks)

Decision: adopt the polymorphic design with exceptions for the hospital group. Reasons from the measurements:

- **Load is small:** 200,000 bills a day is about 2–3 ms of total() calls even with the slowest variant measured (virtual: 146 ms for 10,000,000 bills ⇒ ≈ 3 ms for 200,000). The 20–30% advantage of switch / variant is irrelevant here.
- **Change is the main cost:** two new bill types and three new payers every year across 15 hospitals. With virtual classes each is one new class and the existing loops and tests stay untouched (shown in B4.1); with a switch each one edits every switch in the code and re-opens working code.
- **Errors are rare:** invalid claims in a clean system are a small minority; at 1% a throw costs 12× on a job that takes 2 ms per million bills, i.e. 22 ms – negligible. Exceptions give reasons (ClaimError, UnknownPackageError …) and automatic cleanup by RAII (B2, B4.2).
- **Conditions:** keep total() noexcept and arithmetic-only; virtual destructors everywhere (B1.4); handle bulk imports in which more than about 10% of rows may be invalid with optional/return codes instead of exceptions (B3.3); use integer paise for money.

Alternative Concept and Complexity Analysis – plug-ins described by configuration files

```
FILE rules.cfg:   ER  = BASE + HOURS * RATE        // rule kind and constants only
ALGORITHM Total(bill)
BEGIN
    rule ← rules[bill.typeName]            // lookup by name (hash) or by index
    SWITCH rule.kind                       // interpret the rule
        CASE LINEAR: RETURN rule.a + bill.x * rule.b
        ...
END
```

10,000,000 bills, -O2 (measured)	compiled virtual call	interpreted rule (by name: hash lookup)	interpreted rule (by index)
Time per bill	115.1 ms (11.5 ns)	259.3 ms (25.9 ns)	140.2 ms (14.0 ns)
Space for the rules	code + one shared vtable (a few dozen bytes) per class; 16–32 B per bill object	one Rule = 32 B of data per kind in the file/map; no code	same 32 B per rule

Time: both O(1) per bill; the interpreter is 1.2× (indexed) to 2.3× (by name) slower. Effort and risk: a configuration file can be changed by a non-programmer without recompiling, but can only express the formula shapes the interpreter already knows (a new kind of rule still needs C++), errors surface at run time (typo in a rate, missing field) and must be validated at load; a new C++ class needs a developer, review and a build but is checked by the compiler, tested, and may use any logic. A sensible combination: C++ classes for new kinds of formulas (a few per year), configuration for the rates and package prices that change often – as the GovernmentScheme and package map already do.

Simulation Screenshot

Real terminal run: the completed programs, the three Predict-the-Output programs, the 27-check test run including the extension, the error conditions and the clean sanitizer run.

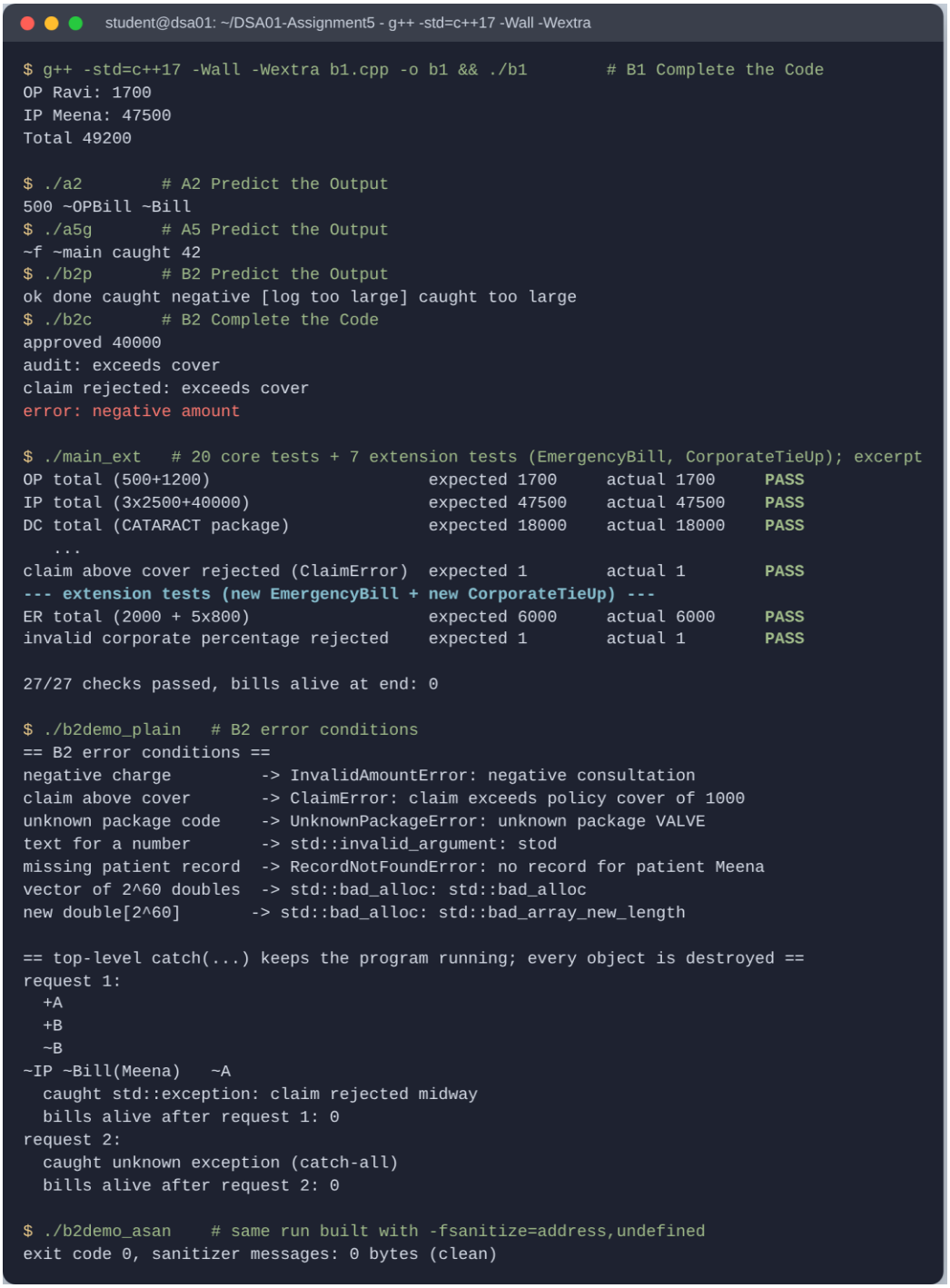


Figure 1 – Terminal run of the Assignment 5 programs.